

On the analysis of OpenFOAM finite volume solution methodologies for linear elasticity

Philip Cardiff

April 2019

Online International Meeting for Users of OpenFOAM
Published on YouTube

On the analysis of OpenFOAM finite volume solution methodologies for linear elasticity

Philip Cardiff

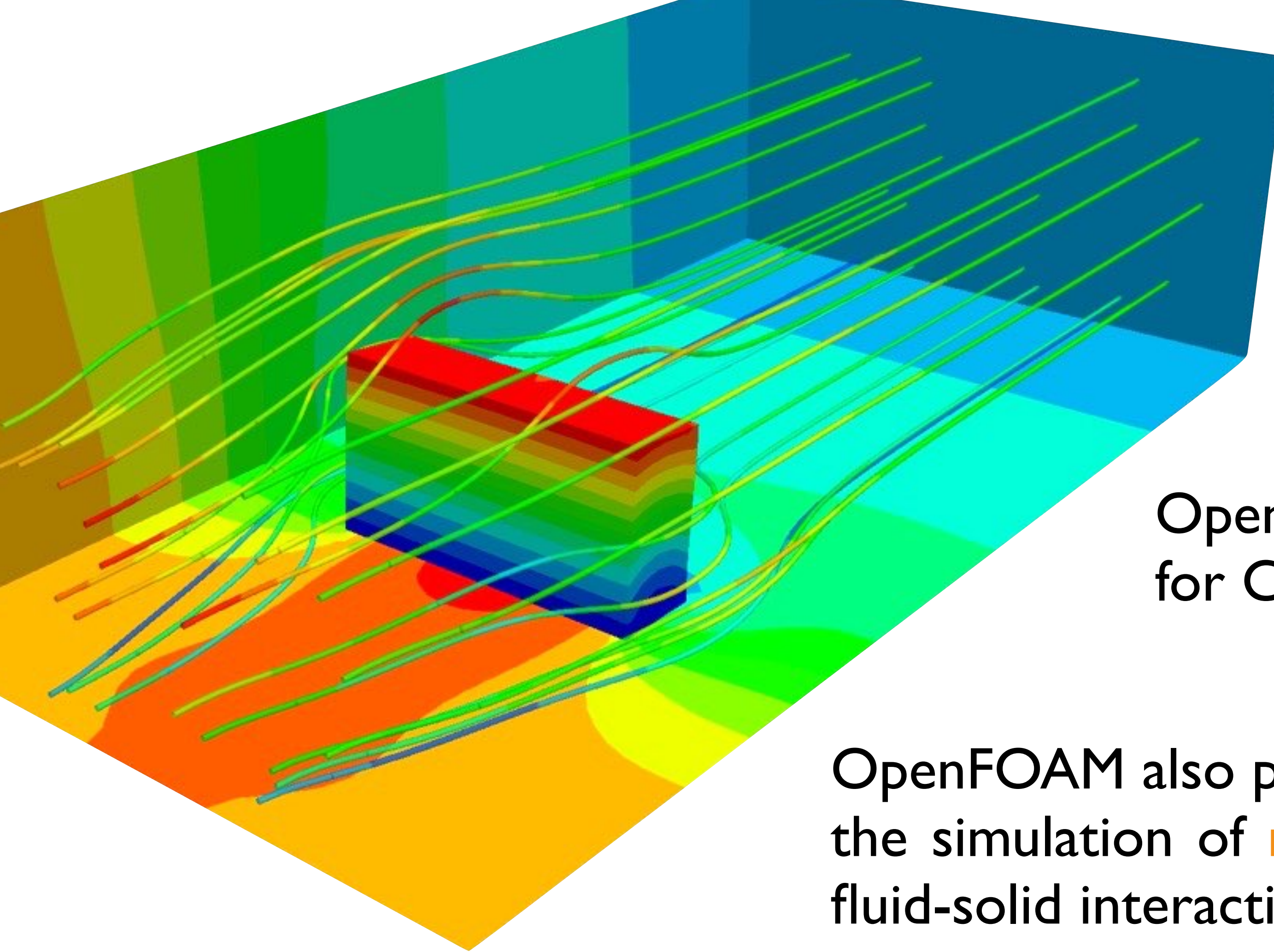


School of Mechanical and Materials Engineering
University College Dublin

A large orange circle is positioned on the left side of the slide, partially cut off by the edge. It contains the text 'Background & Motivation' in white.

Background & Motivation

Background



OpenFOAM is not just an excellent tool for Computational Fluid Dynamics

OpenFOAM also provides a suitable framework for the simulation of **multi-physics problems**, including fluid-solid interaction

The **finite volume method** offers an interesting alternative to the finite element method for **solid mechanics**



Historical
Perspective

The **first reported** application of the finite volume method to **solid mechanics** was Demirdžić et al. in 1988:

Ismet demirdžić
dunja martinović
alojz ivanković

UDK 621.79.791.535.66

numerička simulacija termodeformacionih procesa u zavarenom komadu

numerical simulation of
thermo-deformation
processes in a welded piece

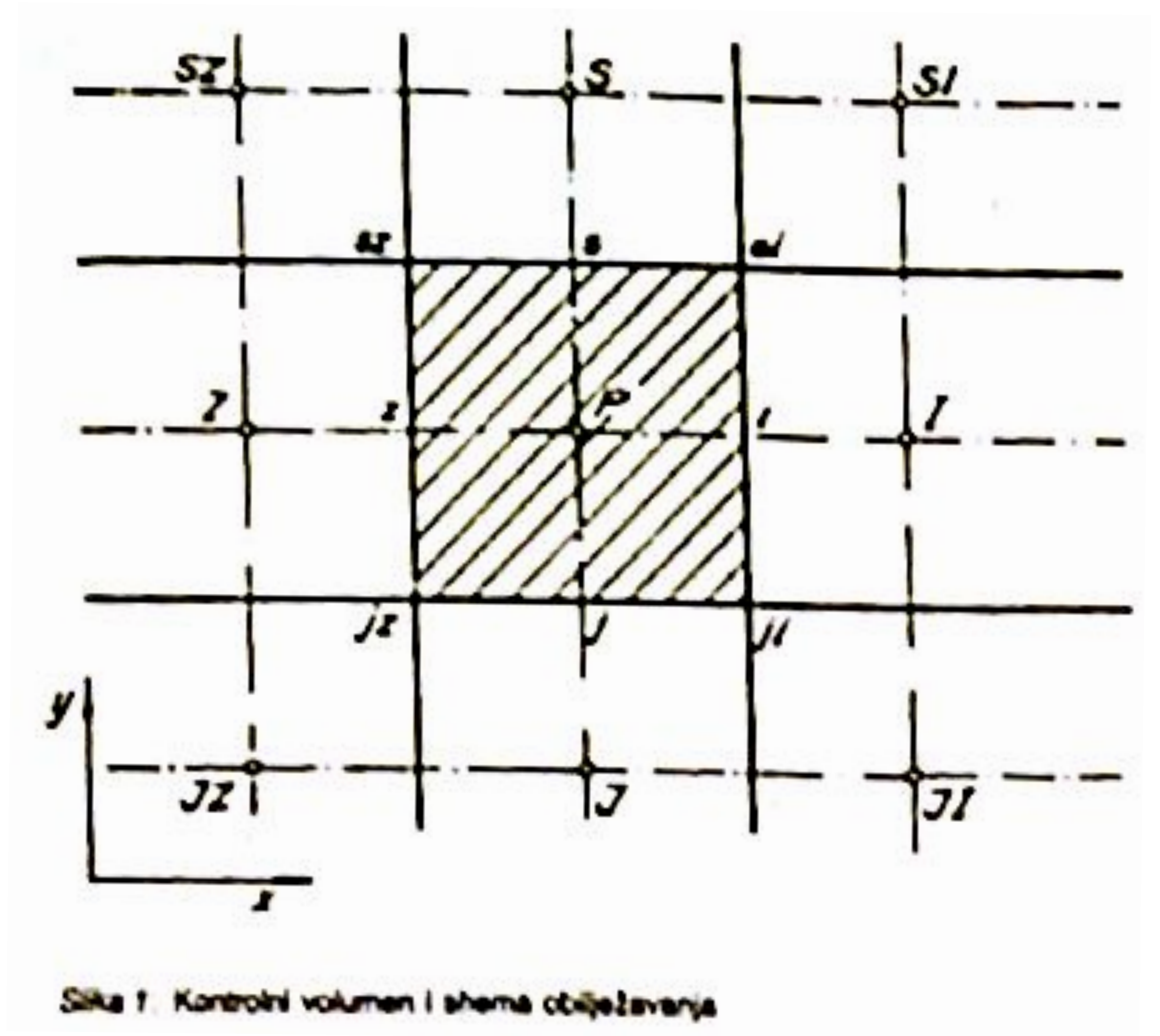
ključne riječi:

- temperaturno polje
- napetosti i deformacije
- numerička analiza
- zavarivanje

U ovom radu je prikazana numerička metoda za analizu termodeformacionih procesa u loku zavarivanja. Metoda se sastoji u simultanom rješavanju jednačbi bilance energije i količine kretanja. Kao rezultat postignu se nestacionarna polja temperature, napetosti i deformacija u radnom komadu.

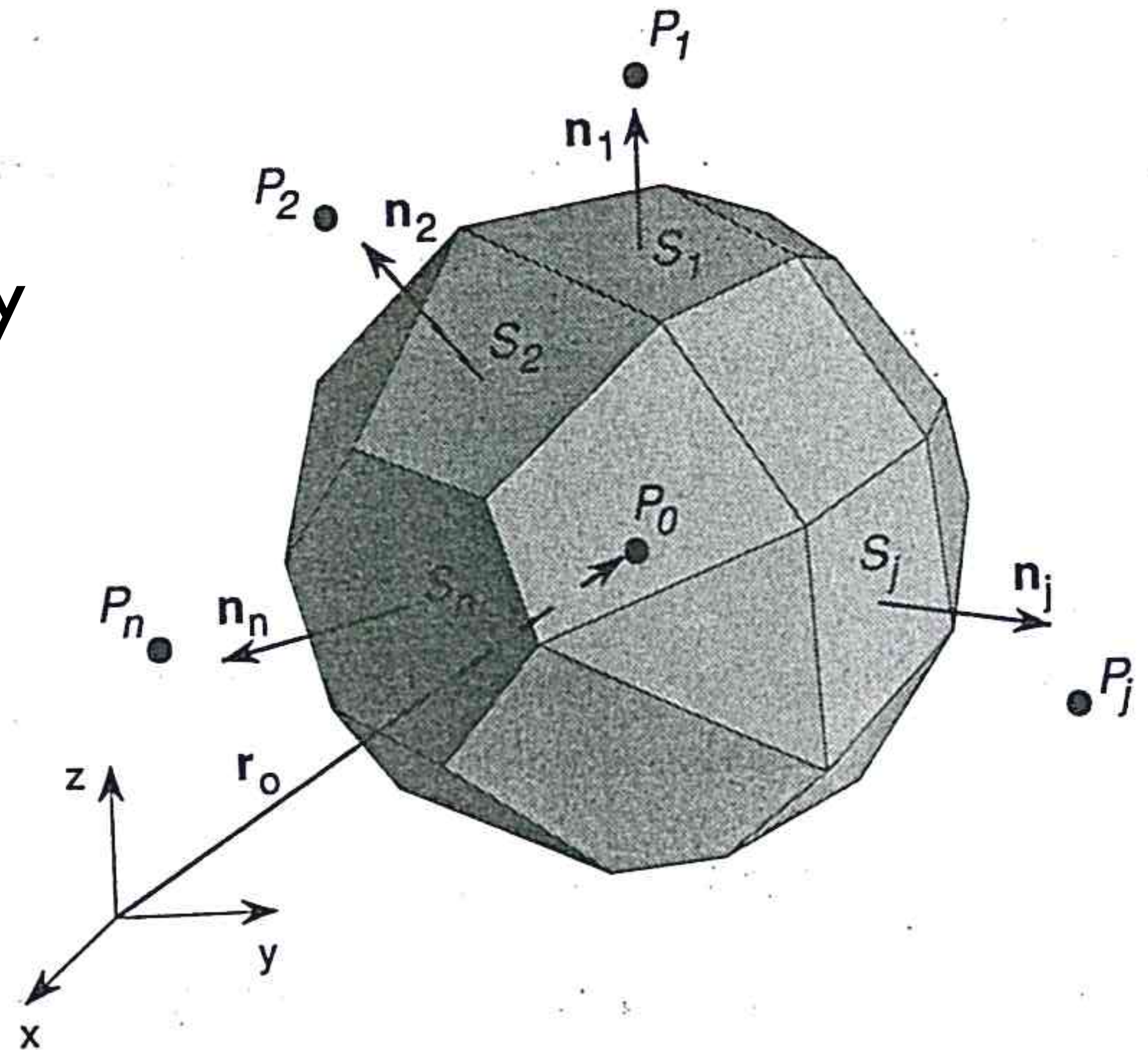
Izvedenim proračunima je pokazano da opisana metoda može poslužiti kao efikasan postupak pri istraživanju termodeformacionih procesa u zavarenom komadu.

The method was initially limited to **2-D structured quadrilaterals** and small-strain thermo-elasticity



The method was initially limited to **2-D structured quadrilaterals** and small-strain thermo-elasticity

and later extended to **3-D unstructured polyhedra**, and a large range of physical phenomena



In the seminal OpenFOAM paper, Weller et al. (1998) developed the first OpenFOAM stress analysis solver, `solidDisplacementFoam`, based on the method of Demirdžić and co-workers

A tensorial approach to using object-oriented tec

H. G. Weller and G. Tabor^{a)}

Department of Mechanical Engineering

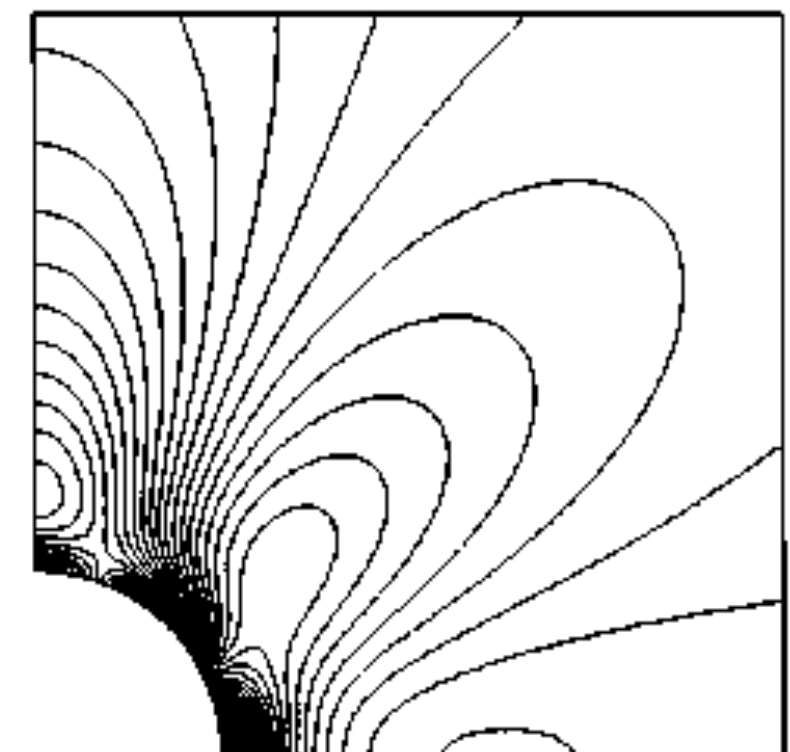
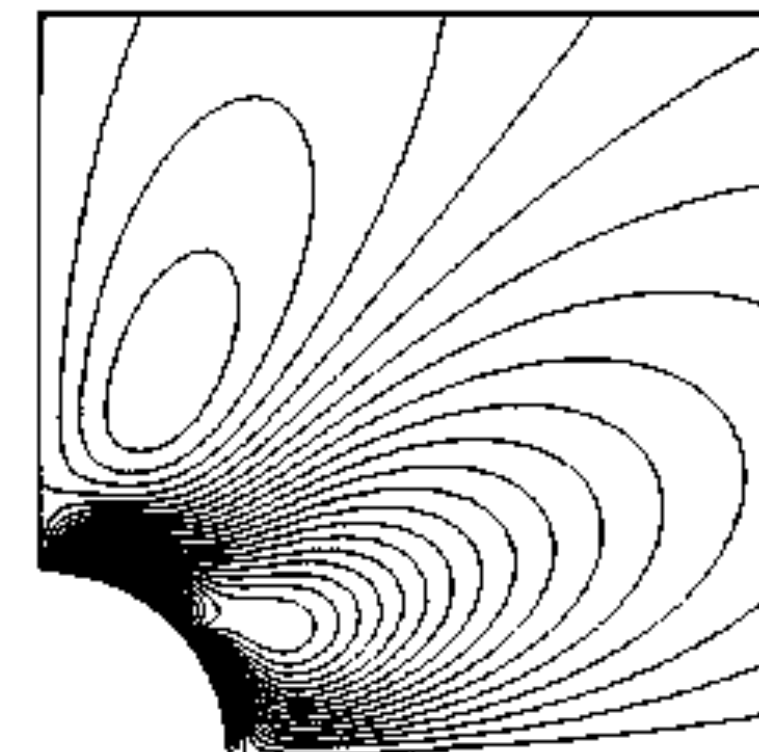
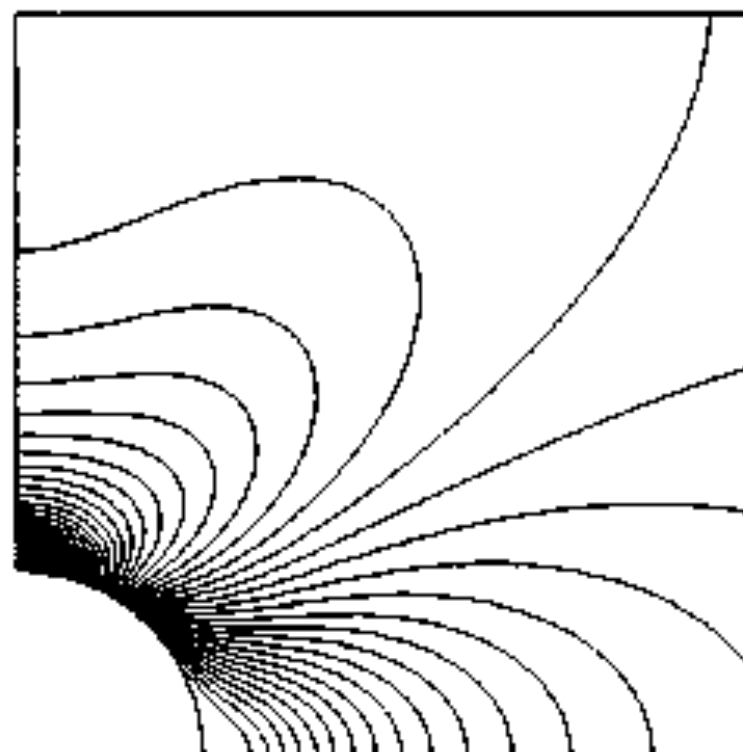
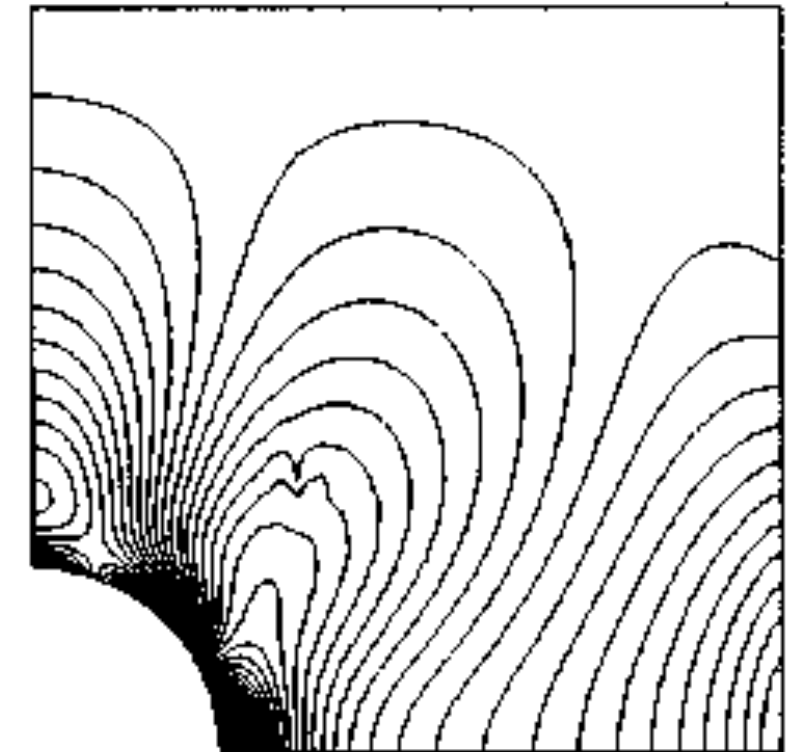
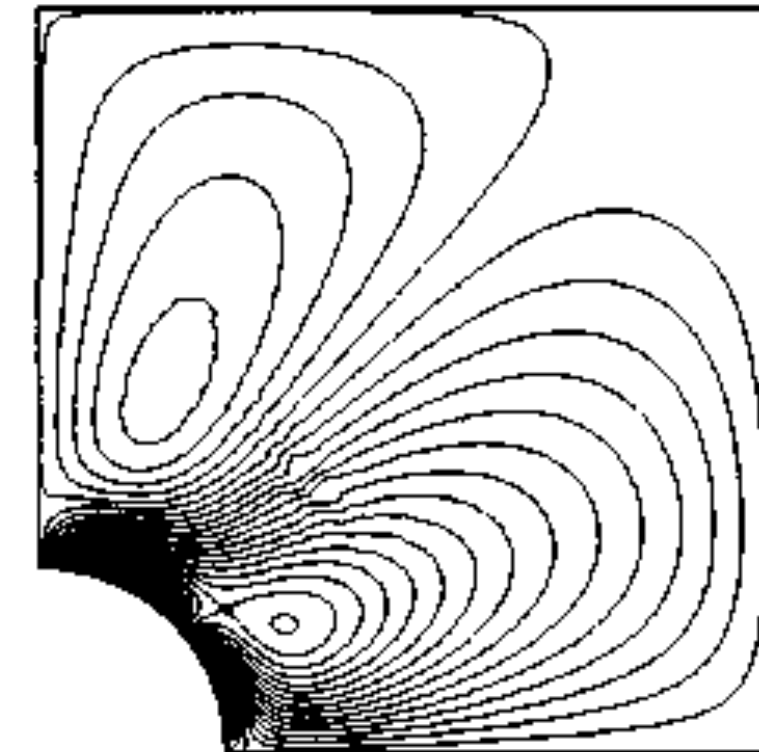
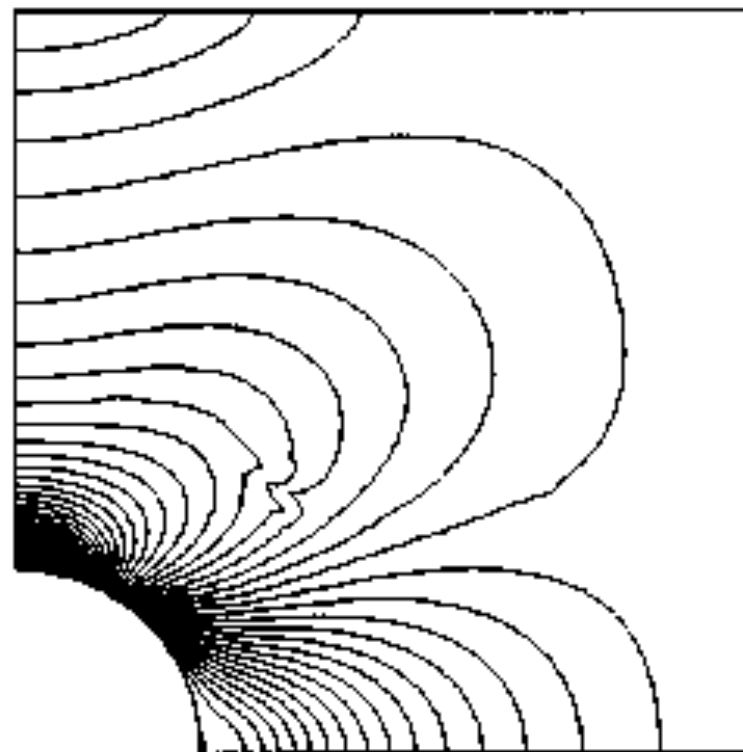
H. Jasak

Computational Dynamics Limited, London

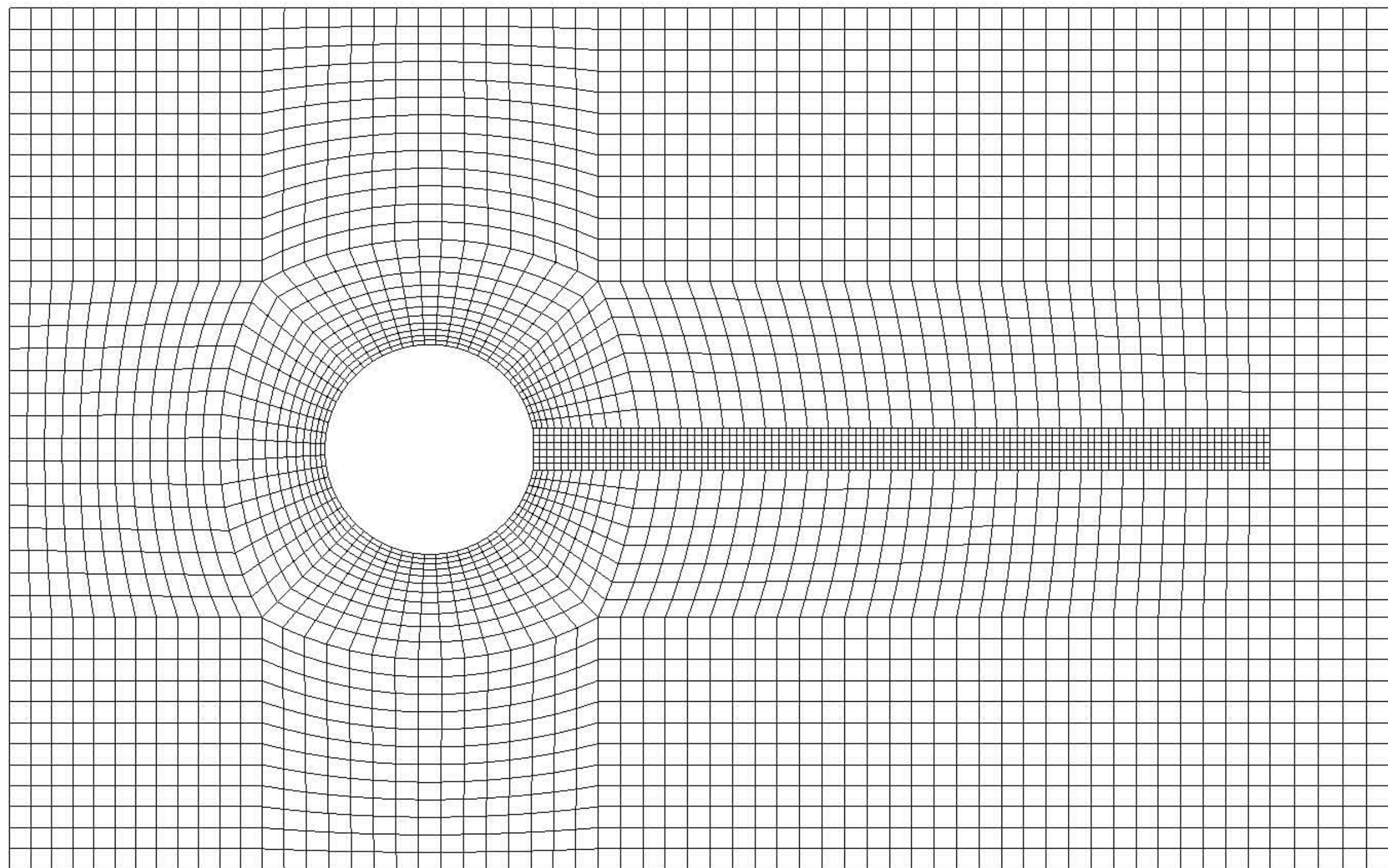
C. Fureby

*Department of Weapons and Protection
S-17290 Stockholm, Sweden*

(Received 1 June 1998; accepted 13 August 1998)



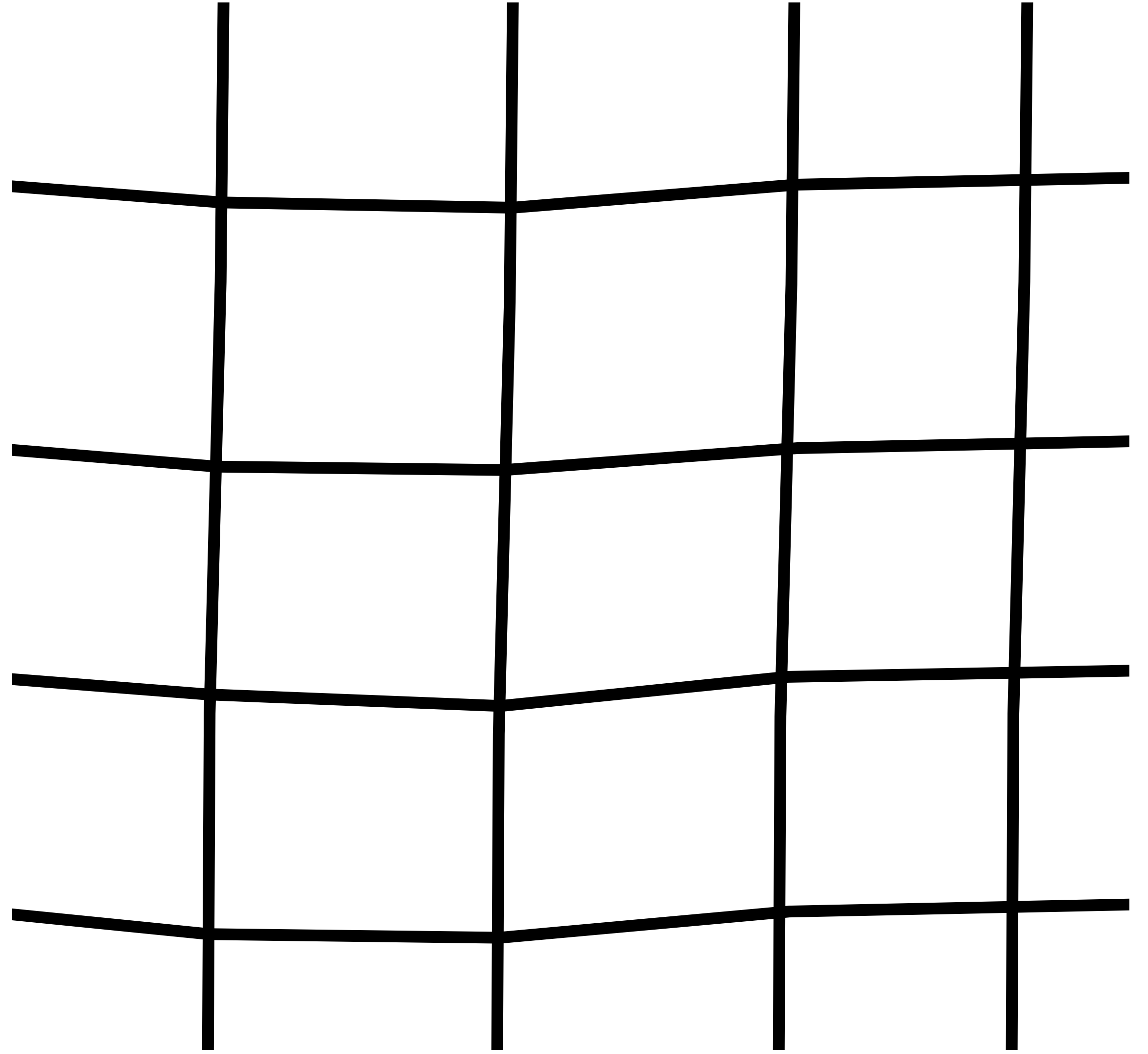
In the seminal OpenFOAM paper, Weller et al. (1998) developed the first OpenFOAM stress analysis solver, `solidDisplacementFoam`, based on the method of Demirdžić and co-workers



Subsequently, developments of solid mechanics in OpenFOAM have included more complex phenomena e.g. `fluid-solid interaction`, elastoplasticity, viscoelasticity, multi-materials, fracture, ...

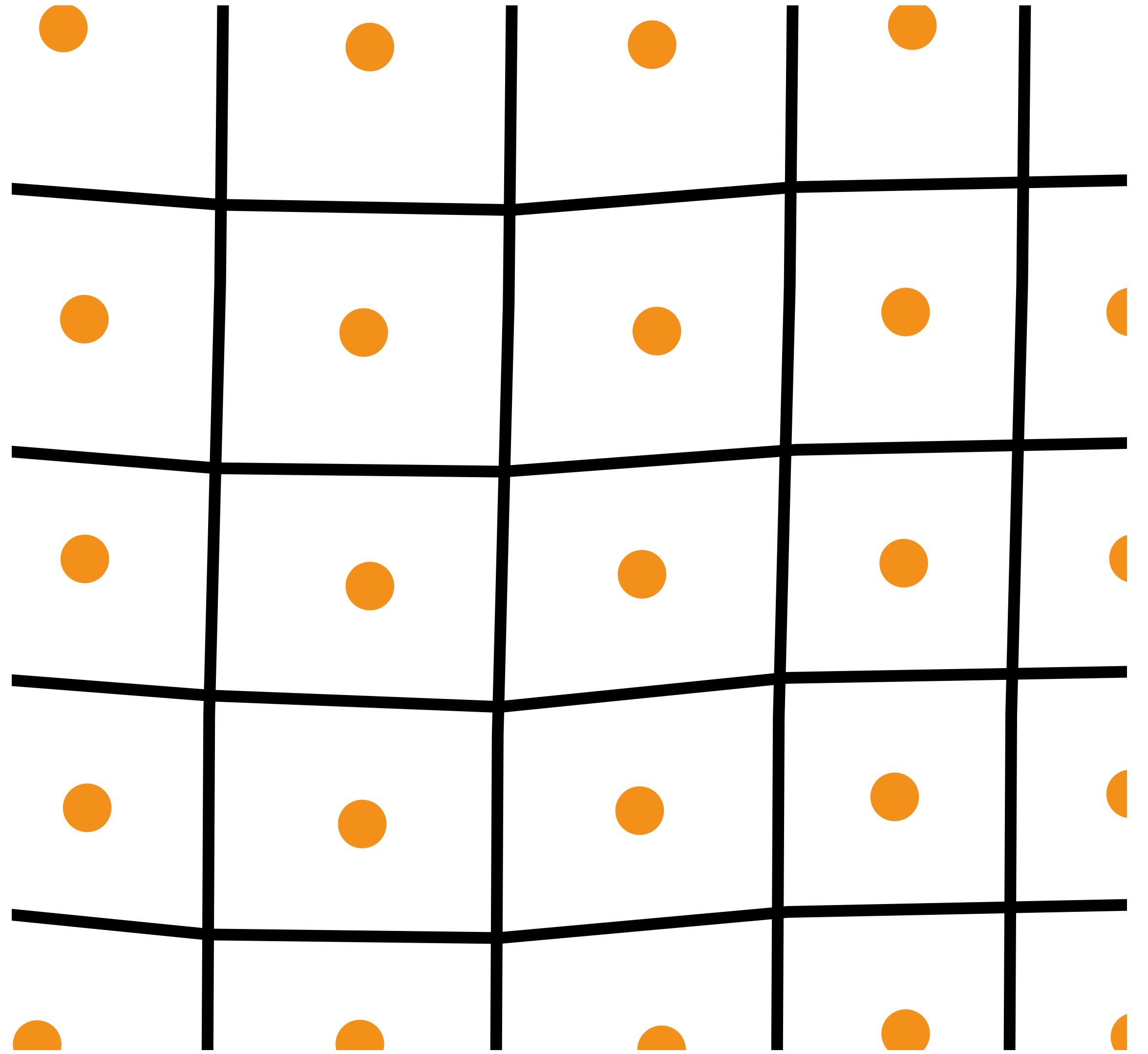
Finite volume approaches for solid mechanics can be **classified** in a number of ways:

- by grid arrangement



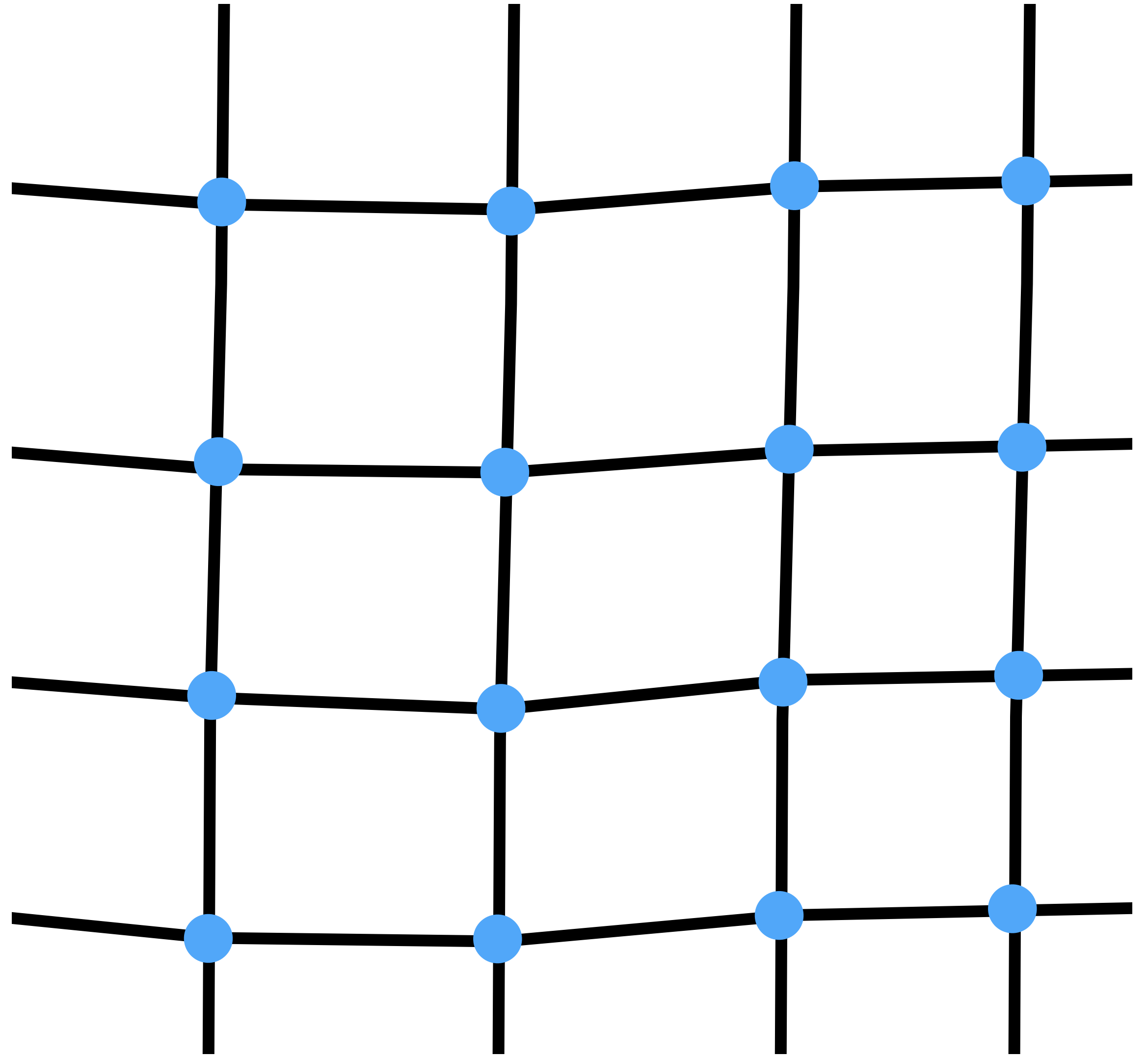
Finite volume approaches for solid mechanics can be classified in a number of ways:

- by grid arrangement
 - cell-centred
 - vertex-centred
 - staggered grid
 - other e.g. face-centred



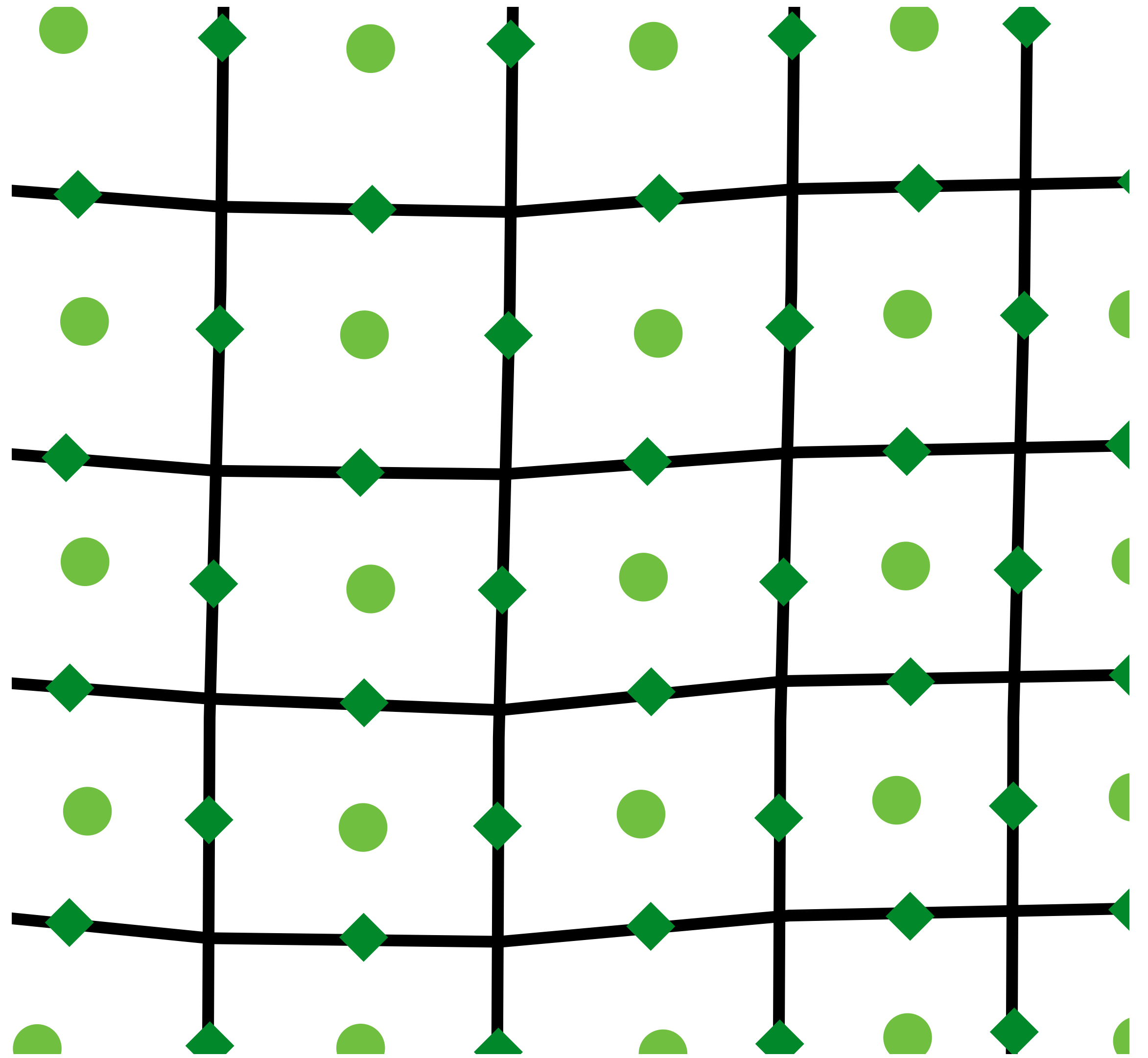
Finite volume approaches for solid mechanics can be classified in a number of ways:

- by grid arrangement
 - cell-centred
 - vertex-centred
 - staggered grid
 - other e.g. face-centred



Finite volume approaches for solid mechanics can be classified in a number of ways:

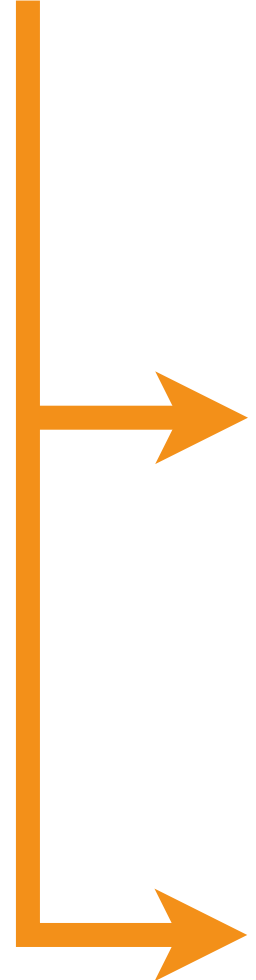
- by grid arrangement
 - cell-centred
 - vertex-centred
 - staggered grid
 - other e.g. face-centred



Finite volume approaches for solid mechanics can be classified in a number of ways:

- by grid arrangement
 - cell-centred
 - vertex-centred
 - staggered grid
 - other e.g. face-centred
- by solution methodology
 - implicit vs explicit

$$\rho \frac{\partial \mathbf{v}}{\partial t} = \nabla \cdot \boldsymbol{\sigma}$$


$$\rho \frac{\mathbf{v}_m - \mathbf{v}_{m-1}}{\Delta t} = \nabla \cdot \boldsymbol{\sigma}_{m-1}$$
$$\rho \frac{\mathbf{v}_m - \mathbf{v}_{m-1}}{\Delta t} = \nabla \cdot \boldsymbol{\sigma}_m$$

Finite volume approaches for solid mechanics can be classified in a number of ways:

- by grid arrangement
 - cell-centred
 - vertex-centred
 - staggered grid
 - other e.g. face-centred
- by solution methodology
 - implicit vs explicit
- by stabilisation method
 - Rhie-Chow vs JST vs Godunov

Stabilisation of pressure oscillation:

large stencil

$$(\nabla p)_f \approx \omega_f (\nabla p)_P + (1 - \omega_f) (\nabla p)_N$$

small stencil

$$(\nabla p)_f \approx \frac{p_N - p_P}{\mathbf{n} \cdot \mathbf{d}}$$

Diffusion term that damps pressure oscillations

$$\mathbb{D}_{\text{RC}} = \frac{p_N - p_P}{\mathbf{n} \cdot \mathbf{d}} - [\omega_f (\nabla p)_P + (1 - \omega_f) (\nabla p)_N]$$

Other diffusion terms

$$\mathbb{D}_{\text{JST}} = \gamma \nabla^2 (\nabla^2 p)$$



Solution Methodologies

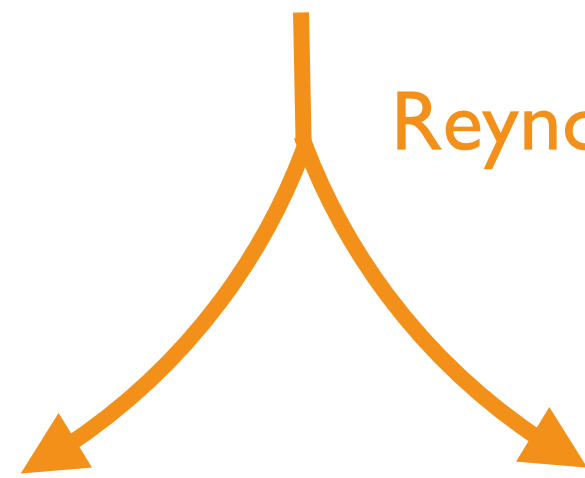
Governing equation

$$\frac{\mathrm{D}}{\mathrm{D}t} \int_{\Omega} \rho \boldsymbol{v} \, \mathrm{d}\Omega = \oint_{\Gamma} \boldsymbol{n} \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma + \int_{\Omega} \rho \boldsymbol{b} \, \mathrm{d}\Omega$$

Governing equation

$$\frac{D}{Dt} \int_{\Omega} \rho \mathbf{v} \, d\Omega = \oint_{\Gamma} \mathbf{n} \cdot \boldsymbol{\sigma} \, d\Gamma + \int_{\Omega} \rho \mathbf{b} \, d\Omega$$

Reynold's transport theorem



$$\int_{\Omega} \frac{\partial}{\partial t} (\rho \mathbf{v}) \, d\Omega + \oint_{\Gamma} \rho \mathbf{v} [\mathbf{n} \cdot (\mathbf{v} - \mathbf{v}_{\Gamma})] \, d\Gamma = \oint_{\Gamma} \mathbf{n} \cdot \boldsymbol{\sigma} \, d\Gamma + \int_{\Omega} \rho \mathbf{b} \, d\Omega$$

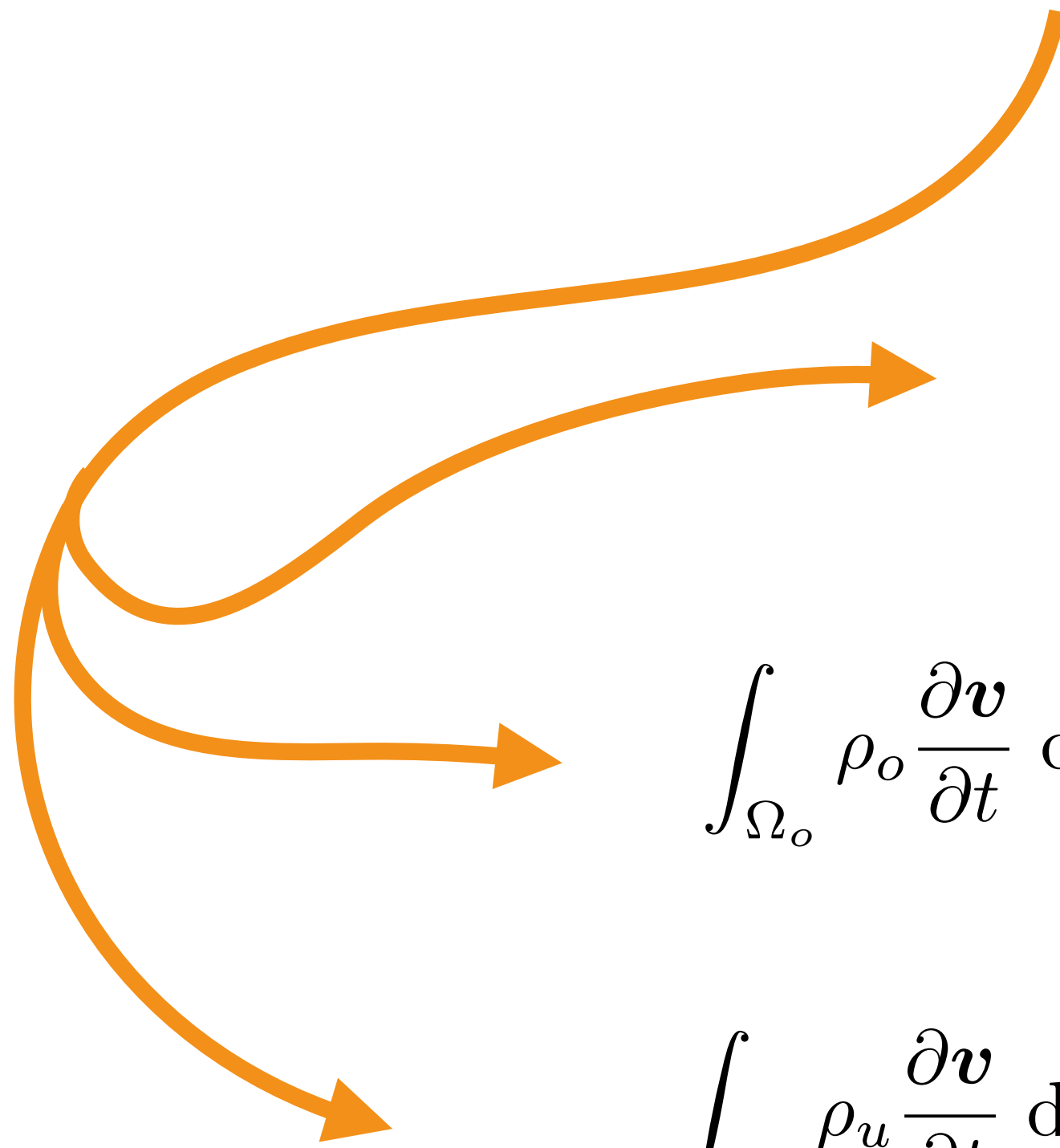
Lagrangian
 $\mathbf{v} - \mathbf{v}_{\Gamma} = 0$



$$\boldsymbol{\sigma} = f(\mathbf{v}, \mathbf{u}, \Delta \mathbf{u}, \dots)$$



$$\int_{\Omega} \frac{\partial \rho \mathbf{v}}{\partial t} \, \mathrm{d}\Omega = \oint_{\Gamma} \mathbf{n} \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma + \int_{\Omega} \rho \mathbf{b} \, \mathrm{d}\Omega$$



linear geometry

$$\int_{\Omega_o} \rho_o \frac{\partial \mathbf{v}}{\partial t} \, \mathrm{d}\Omega_o = \oint_{\Gamma_o} \mathbf{n}_o \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma_o + \int_{\Omega_o} \rho_o \mathbf{b} \, \mathrm{d}\Omega_o$$

nonlinear geometry total Lagrangian

$$\int_{\Omega_o} \rho_o \frac{\partial \mathbf{v}}{\partial t} \, \mathrm{d}\Omega_o = \oint_{\Gamma_o} \left(J \mathbf{F}^{-T} \cdot \mathbf{n}_o \right) \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma_o + \int_{\Omega_o} \rho_o \mathbf{b} \, \mathrm{d}\Omega_o$$

nonlinear geometry updated Lagrangian

$$\int_{\Omega_u} \rho_u \frac{\partial \mathbf{v}}{\partial t} \, \mathrm{d}\Omega_u = \oint_{\Gamma_u} \left(j \mathbf{f}^{-T} \cdot \mathbf{n}_u \right) \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma_u + \int_{\Omega_u} \rho_u \mathbf{b} \, \mathrm{d}\Omega_u$$

$$\mathbf{F} = \mathbf{I} + (\nabla \mathbf{u})^T$$

$$J = \det[\mathbf{F}]$$

$$\mathbf{f} = \mathbf{I} + (\nabla[\Delta \mathbf{u}])^T$$

$$j = \det[\mathbf{f}]$$

$$\int_{\Omega_o} \rho_o \frac{\partial \boldsymbol{v}}{\partial t} \, \mathrm{d}\Omega_o = \oint_{\Gamma_o} \boldsymbol{n}_o \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma_o + \int_{\Omega_o} \rho_o \boldsymbol{b} \, \mathrm{d}\Omega_o$$



$$\int_{\Omega_o} \rho_o \frac{\partial^2 \boldsymbol{u}}{\partial t^2} \, \mathrm{d}\Omega_o = \oint_{\Gamma_o} \boldsymbol{n}_o \cdot \boldsymbol{\sigma} \, \mathrm{d}\Gamma_o + \int_{\Omega_o} \rho_o \boldsymbol{b} \, \mathrm{d}\Omega_o$$



$$\int_{\Omega_o} \rho_o \frac{\partial^2 \boldsymbol{u}}{\partial t^2} \, \mathrm{d}\Omega_o = \oint_{\Gamma_o} \boldsymbol{n}_o \cdot [\mu \boldsymbol{\nabla} \boldsymbol{u} + \mu (\boldsymbol{\nabla} \boldsymbol{u})^T + \lambda \operatorname{tr}(\boldsymbol{\nabla} \boldsymbol{u}) \mathbf{I}] \, \mathrm{d}\Gamma_o + \int_{\Omega_o} \rho_o \boldsymbol{b} \, \mathrm{d}\Omega_o$$

$$\int_{\Omega_o} \rho_o \frac{\partial^2 \mathbf{u}}{\partial t^2} d\Omega_o = \oint_{\Gamma_o} \mathbf{n}_o \cdot [\mu \nabla \mathbf{u} + \mu (\nabla \mathbf{u})^T + \lambda \text{tr}(\nabla \mathbf{u}) \mathbf{I}] d\Gamma_o + \int_{\Omega_o} \rho_o \mathbf{b} d\Omega_o$$

We will consider a selection of differing solution methodologies:

- **implicit - segregated**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: interpolate gradients from cell-centres
 - ▶ *stabilisation*: Rhie-Chow or JST stabilisation
- **implicit - block-coupled**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: calculate normal/tangential gradient at face
 - ▶ *stabilisation*: none
- **explicit**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: interpolate gradients from cell-centres
 - ▶ *stabilisation*: JST stabilisation

- implicit - **segregated**

- ▶ *primitive variable*: total displacement
- ▶ *face gradient discretisation*: interpolate gradients from cell-centres
- ▶ *stabilisation*: Rhie-Chow or JST stabilisation

```
do
{
    solve
    (
        rho*fvm::d2dt2(U)
        == fvm::laplacian(mu, U)
        + fvc::div(mu*gradU.T() + lambda*tr(gradU)*I)
        + rho*gravity
        + stabilisation
    );

    gradU = fvc::grad(U);
    stabilisation = alpha*(fvc::laplacian(mu, U) - fvc::div(mu*gradU)); // Rhie-Chow
    //stabilisation = -alpha*fvc::laplacian(mesh.magSf(), fvc::laplacian(mu, U)); // JST
}
while (!converged);
```

Typical values for the stabilisation scale factor (alpha) are 1.0 for Rhie-Chow and 0.001 for JST.

- implicit - **segregated**

- ▶ *primitive variable*: total displacement
- ▶ *face gradient discretisation*: interpolate gradients from cell-centres
- ▶ *stabilisation*: Rhie-Chow or JST stabilisation

```
do
{
    solve
    (
        rho*fvm::d2dt2(U)
    == fvm::laplacian(2*mu + lambda, U)
    - fvc::laplacian(mu + lambda, U)
    + fvc::div(mu*gradU.T() + lambda*tr(gradU)*I)
    + rho*gravity
    + stabilisation
    );

    gradU = fvc::grad(U);
    stabilisation = alpha*(fvc::laplacian(mu, U) - fvc::div(mu*gradU)); // Rhie-Chow
    //stabilisation = -alpha*fvc::laplacian(mesh.magSf(), fvc::laplacian(mu, U)); // JST
}
while (!converged);
```

Jasak & Weller “over-relaxed” approach

$$\int_{\Omega_o} \rho_o \frac{\partial^2 \mathbf{u}}{\partial t^2} d\Omega_o = \oint_{\Gamma_o} \mathbf{n}_o \cdot [\mu \nabla \mathbf{u} + \mu (\nabla \mathbf{u})^T + \lambda \text{tr}(\nabla \mathbf{u}) \mathbf{I}] d\Gamma_o + \int_{\Omega_o} \rho_o \mathbf{b} d\Omega_o$$

We will consider a selection of differing solution methodologies:

- **implicit - segregated**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: interpolate gradients from cell-centres
 - ▶ *stabilisation*: Rhie-Chow or JST stabilisation
- **implicit - block-coupled**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: calculate normal/tangential gradient at face
 - ▶ *stabilisation*: none
- **explicit**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: interpolate gradients from cell-centres
 - ▶ *stabilisation*: JST stabilisation

- implicit - **block-coupled**

- ▶ *primitive variable*: total displacement
- ▶ *face gradient discretisation*: calculate normal/tangential gradient at face
- ▶ *stabilisation*: none

```
solve
(
    rho*fvm::d2dt2(U)
== BlockFvm::laplacian(mu, U)
+ BlockFvm::laplacianTranspose(mu, U)
+ BlockFvm::laplacianTrace(lambda, U)
+ rho*gravity
);

...    // actual code has a few more steps
```

$$\int_{\Omega_o} \rho_o \frac{\partial^2 \mathbf{u}}{\partial t^2} d\Omega_o = \oint_{\Gamma_o} \mathbf{n}_o \cdot [\mu \nabla \mathbf{u} + \mu (\nabla \mathbf{u})^T + \lambda \text{tr}(\nabla \mathbf{u}) \mathbf{I}] d\Gamma_o + \int_{\Omega_o} \rho_o \mathbf{b} d\Omega_o$$

We will consider a selection of differing solution methodologies:

- **implicit - segregated**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: interpolate gradients from cell-centres
 - ▶ *stabilisation*: Rhie-Chow or JST stabilisation
- **implicit - block-coupled**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: calculate normal/tangential gradient at face
 - ▶ *stabilisation*: none
- **explicit**
 - ▶ *primitive variable*: total displacement
 - ▶ *face gradient discretisation*: interpolate gradients from cell-centres
 - ▶ *stabilisation*: JST stabilisation

- **explicit**

- ▶ *primitive variable*: total displacement
- ▶ *face gradient discretisation*: interpolate gradients from cell-centres
- ▶ *stabilisation*: JST stabilisation

```
// Update velocity
V = V.oldTime() + runTime.deltaT()*a.oldTime();

// Update displacement
U = U.oldTime() + runTime.deltaT()*V;

// Enforce displacement boundary conditions
U.correctBoundaryConditions();

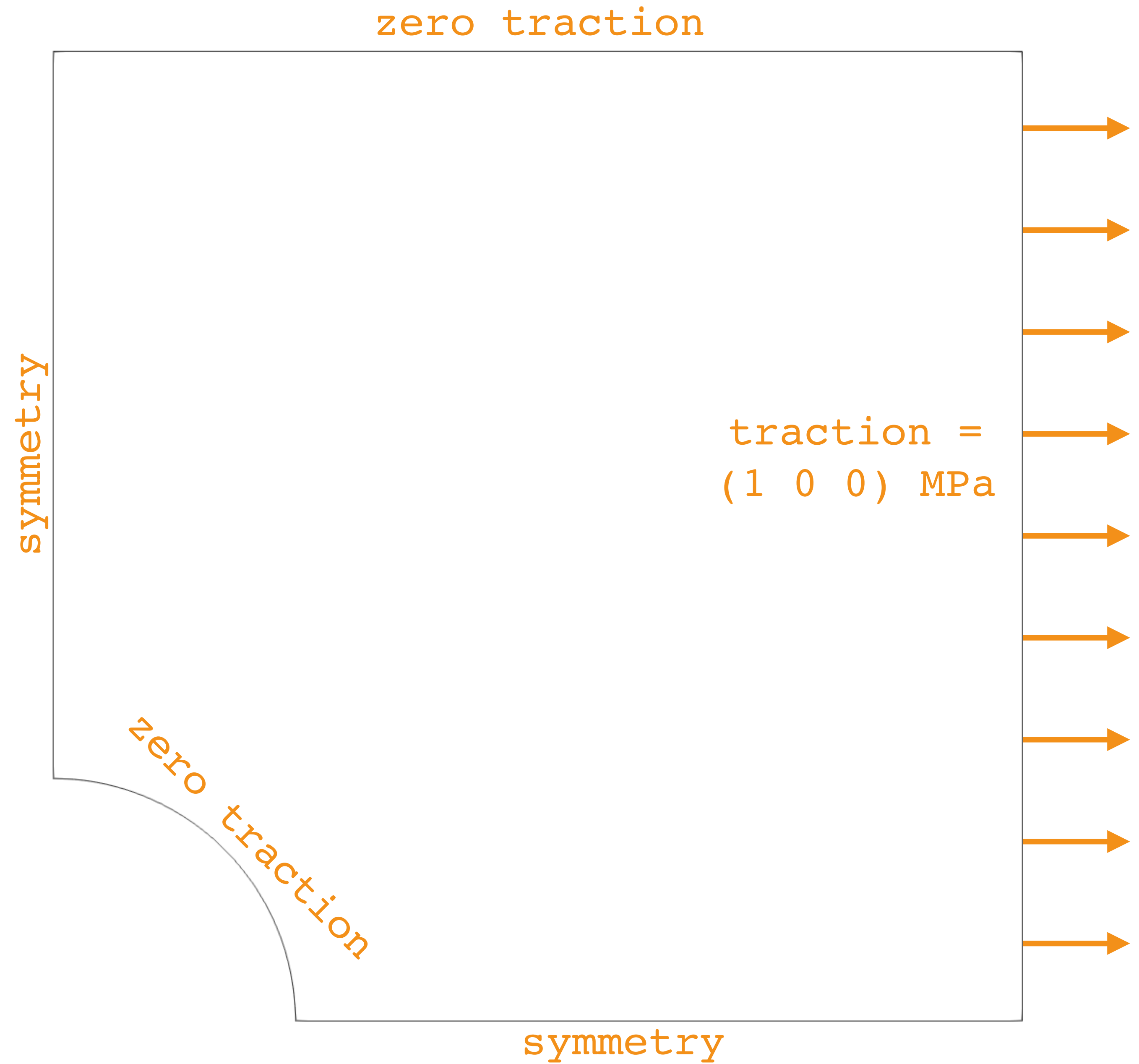
// Update stress
sigma = 2*mu*symm(fvc::grad(U)) + lambda*tr(fvc::grad(U))*I;

// Calculate JST stabilisation
stabilisation = -alpha*fvc::laplacian(mesh.magSf(), fvc::laplacian(mu, V))

// Update acceleration
a = (fvc::div(sigma) + stabilisation)/rho + gravity;
```

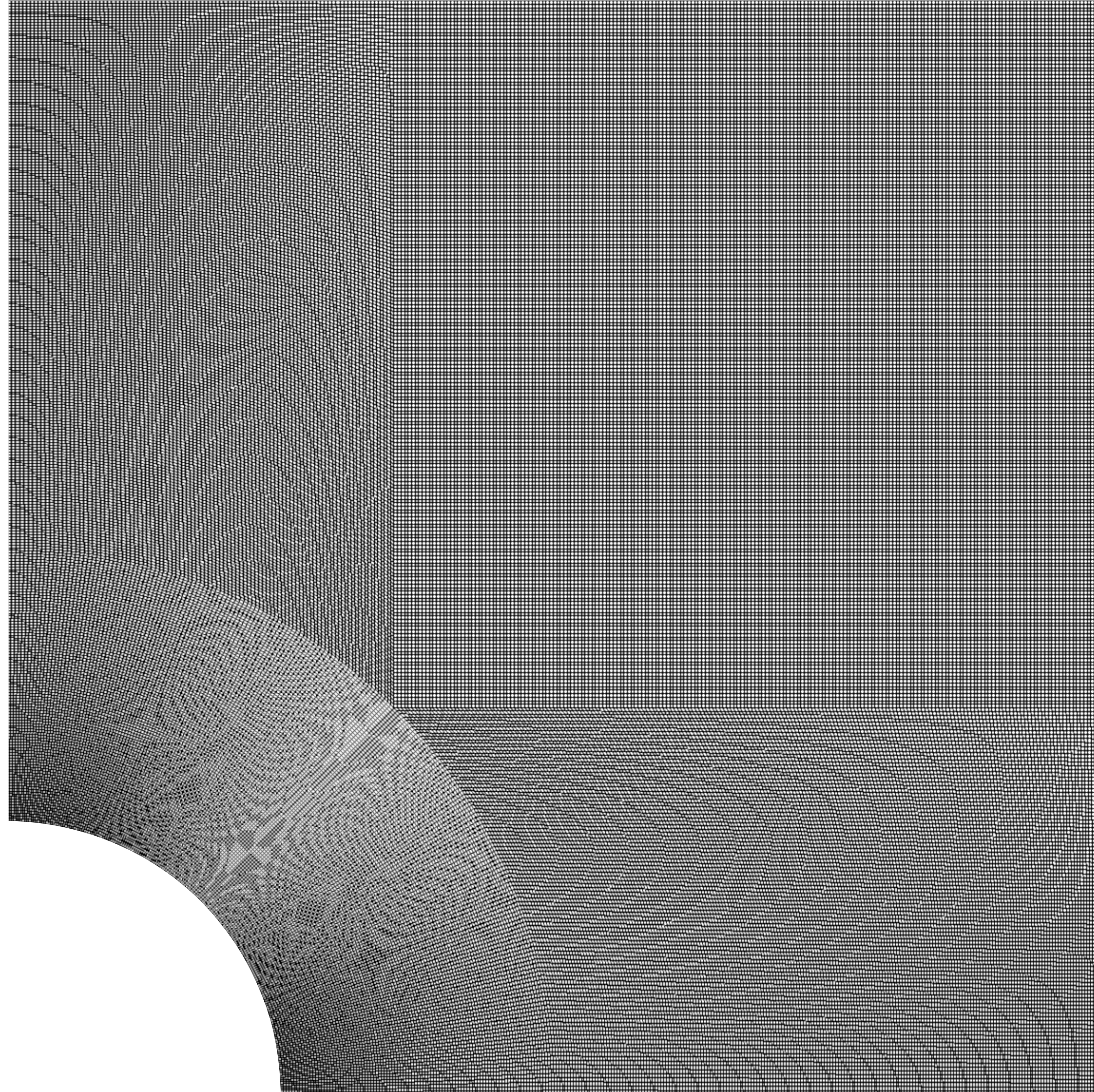
Example Cases

plateHole



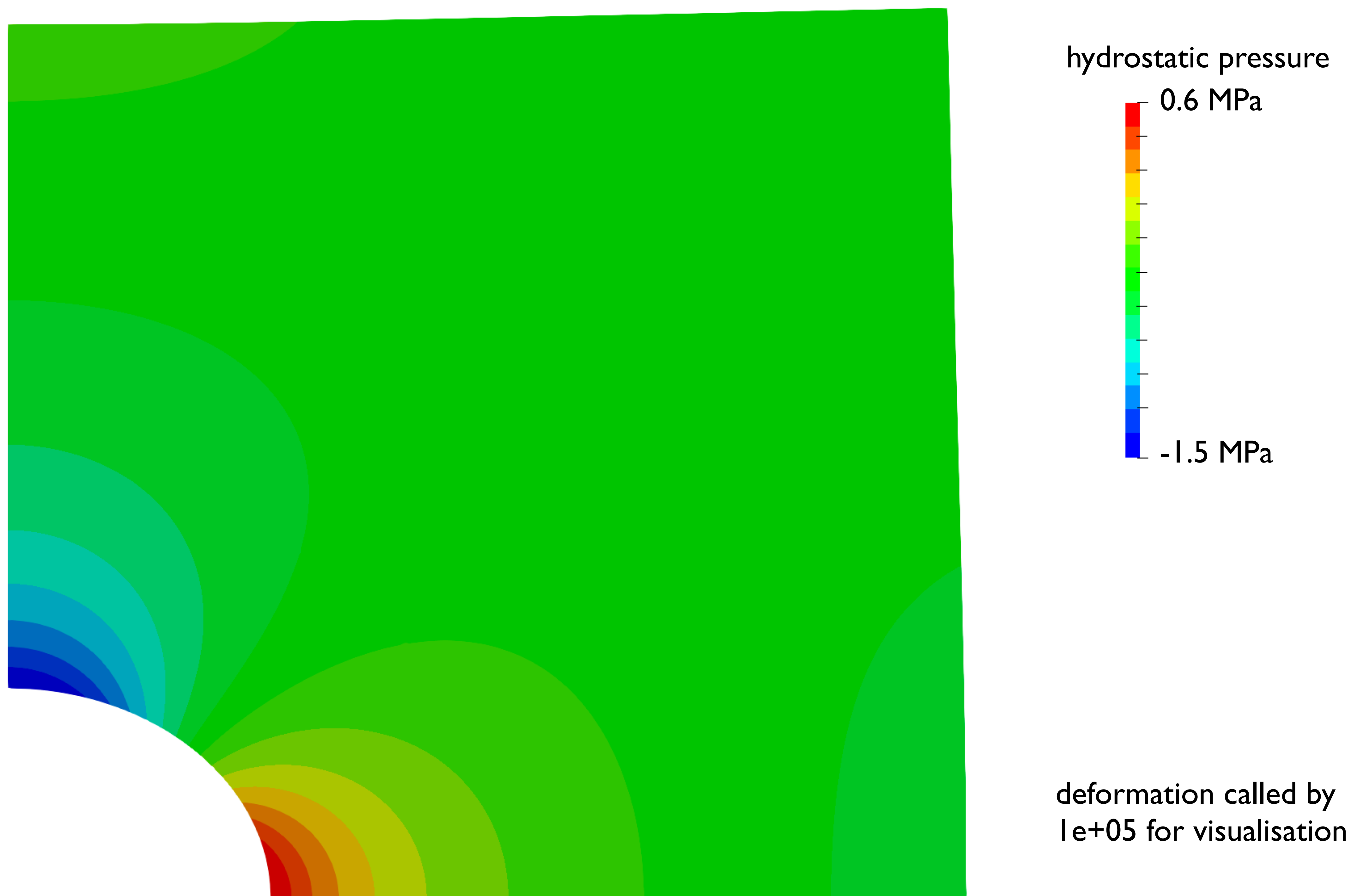
plateHole

100,000 hexahedral cells



plateHole

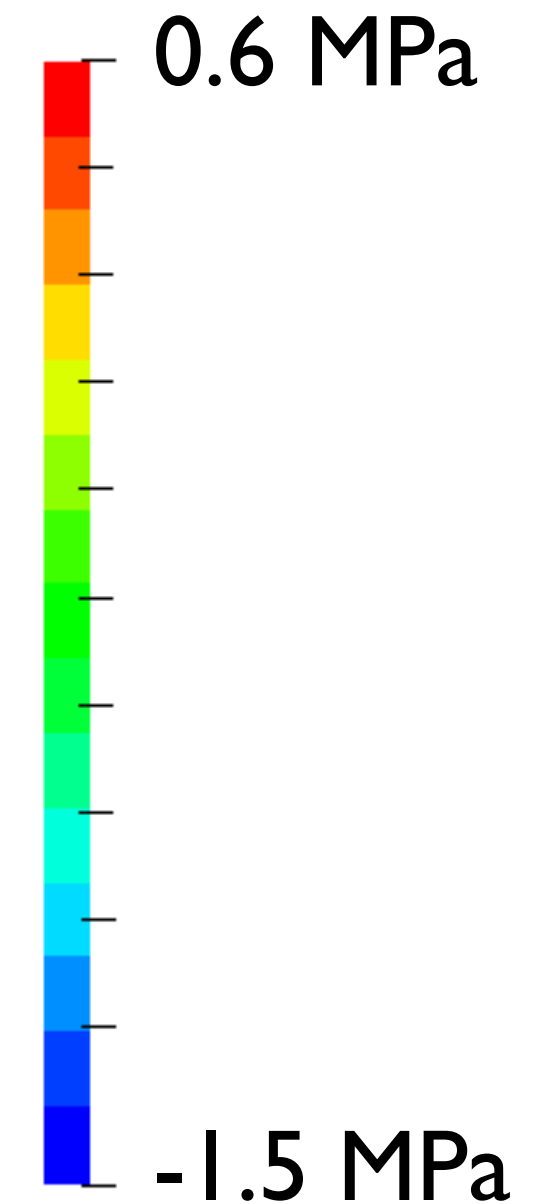
- ▶ **steady-state** : 1 time-step, no inertial effects
 - implicit, segregated, interpolated gradients, Rhie-Chow stabilisation
 - implicit, coupled, direct gradients, no stabilisation, direct linear solver
- ▶ **transient** : end-time = 0.01 s, $\Delta T = 1.066e-05$ s
 - explicit, interpolated gradients, JST stabilisation
 - implicit, segregated, interpolated gradients, Rhie-Chow stabilisation
 - implicit, segregated, interpolated gradients, RC stabilisation, $\times 10 \Delta T$
 - implicit, coupled, direct gradients, no stabilisation, iterative linear solver



At this mesh resolution, all approaches give the same solution

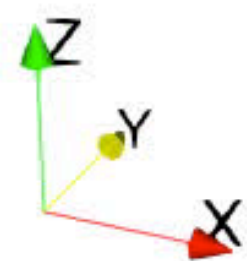
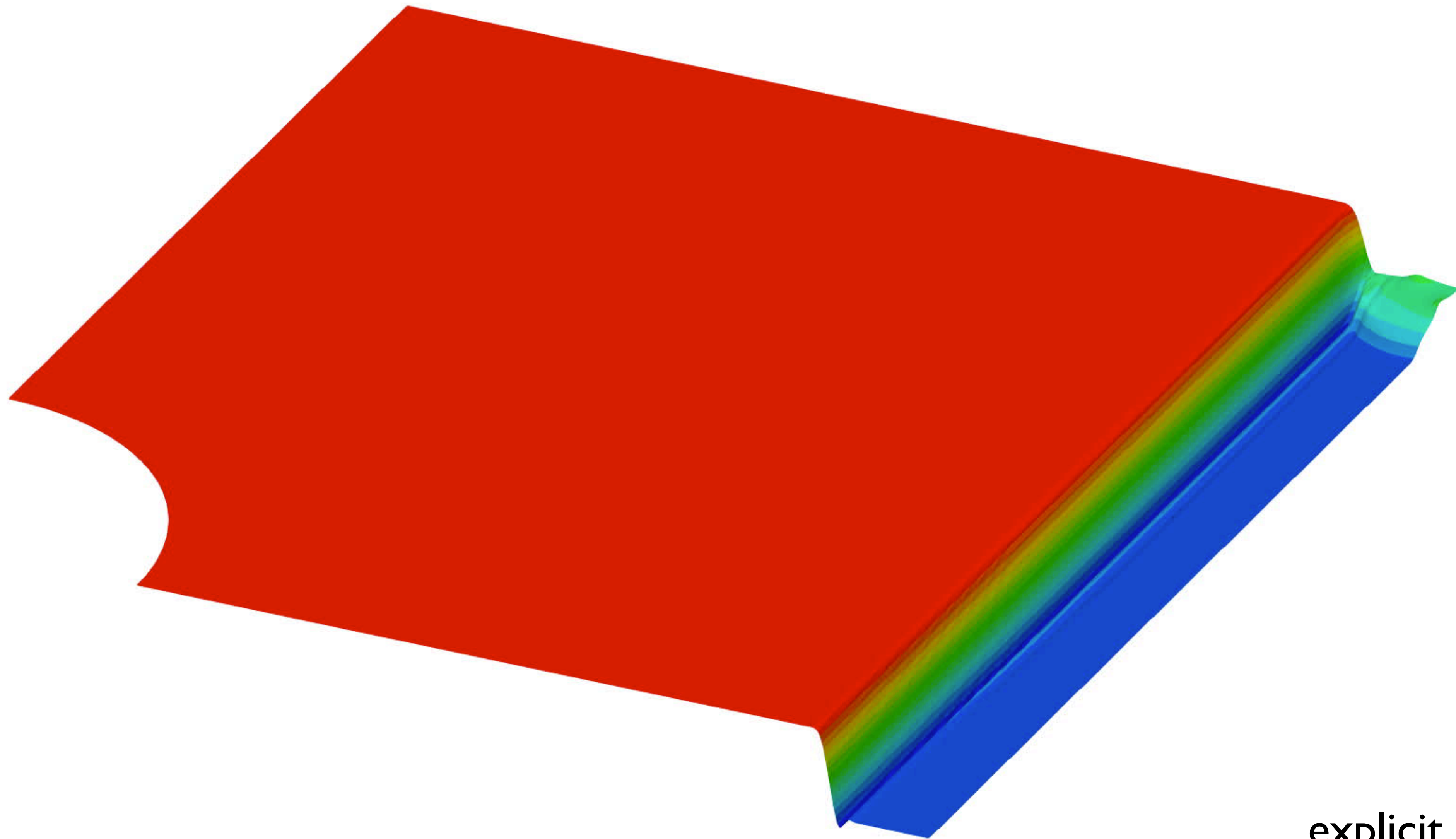
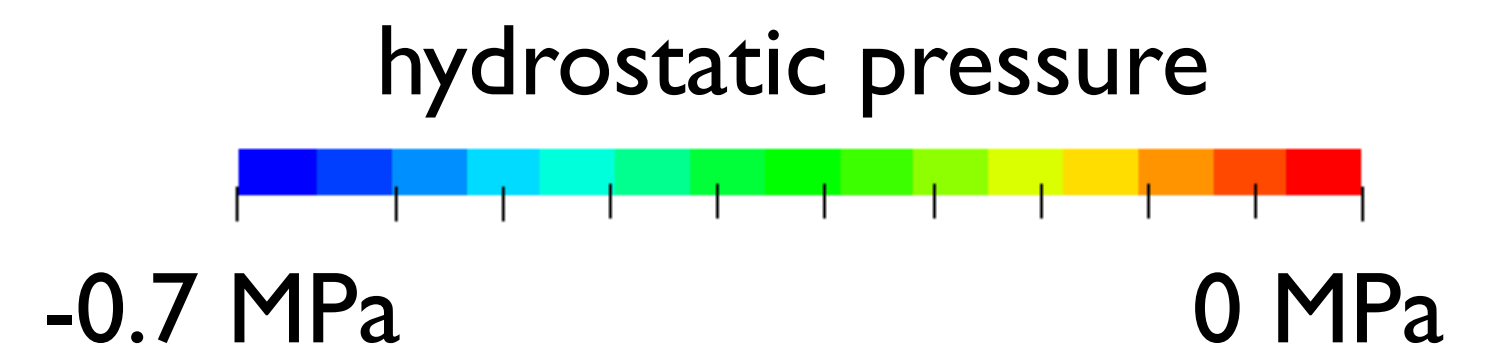
- implicit, **segregated**, interpolated gradients, RC stabilisation
 - outer iterations: 126
 - time: 187 s
- implicit, **coupled**, direct gradients, no stabilisation, direct linear solver
 - outer iterations: 1 (system is linear)
 - time: 23 s

hydrostatic pressure



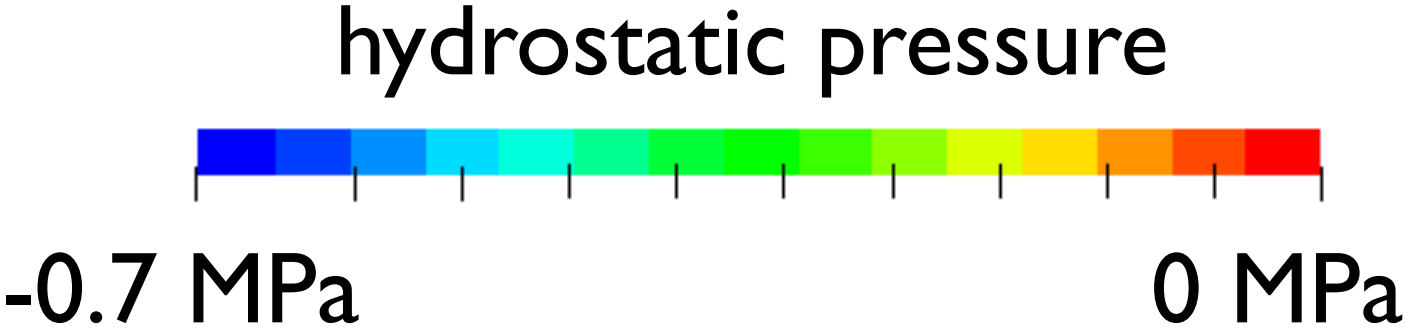
deformation called by
1e+04 for visualisation

Time: 1.002 ms

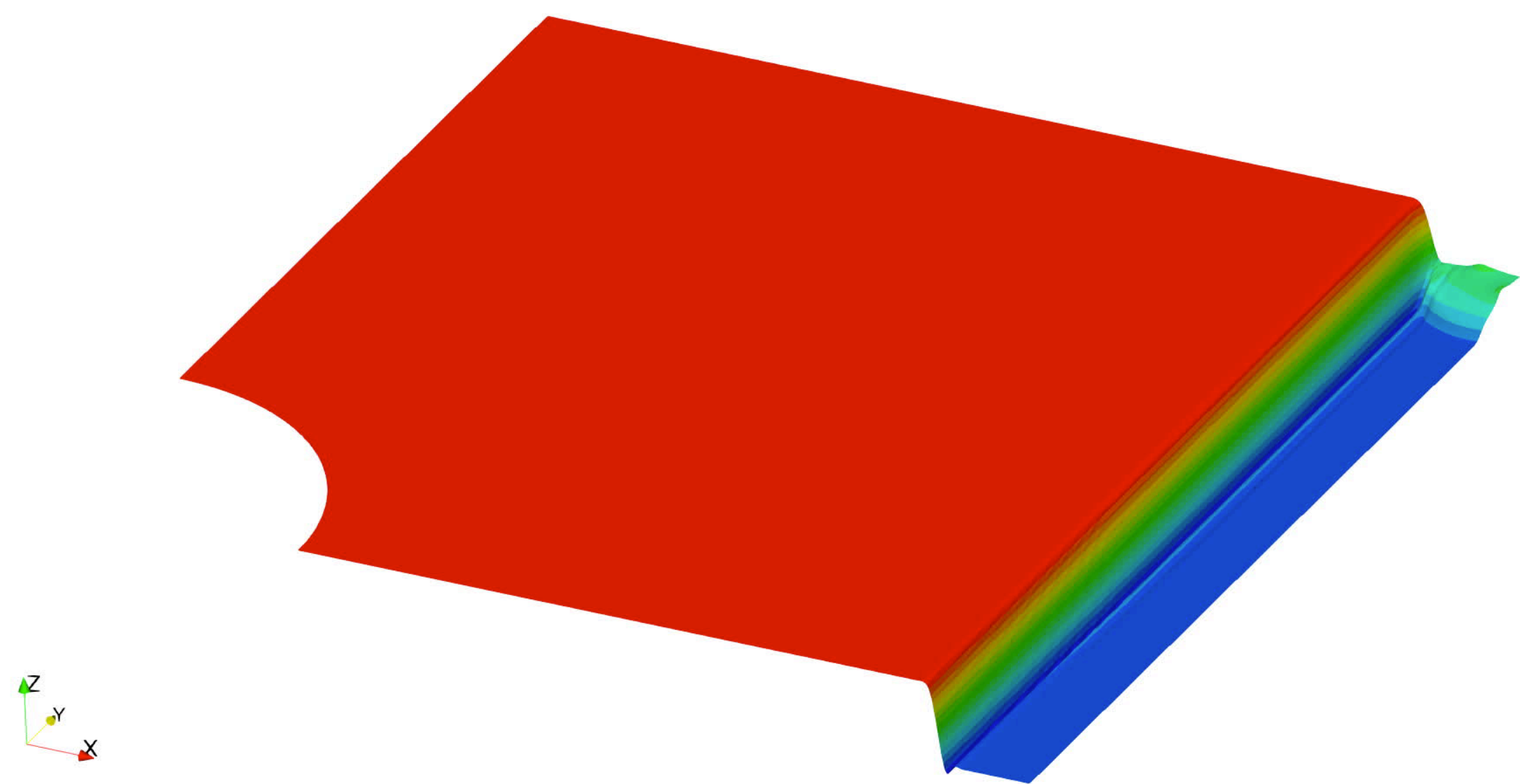


geometry deformed in the Z direction by $2.75e-07 \times \text{pressure}$

explicit

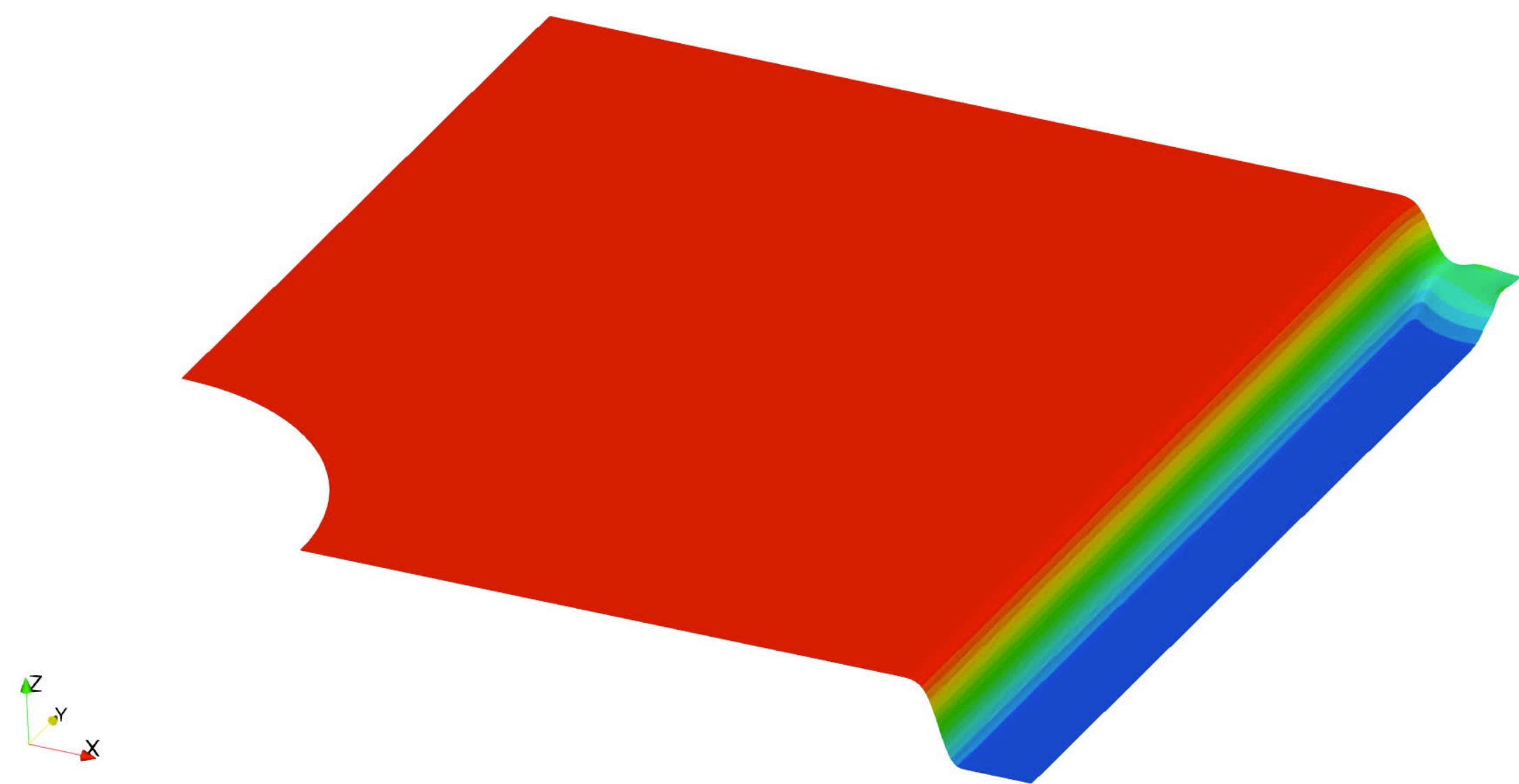


Time: 1.002 ms

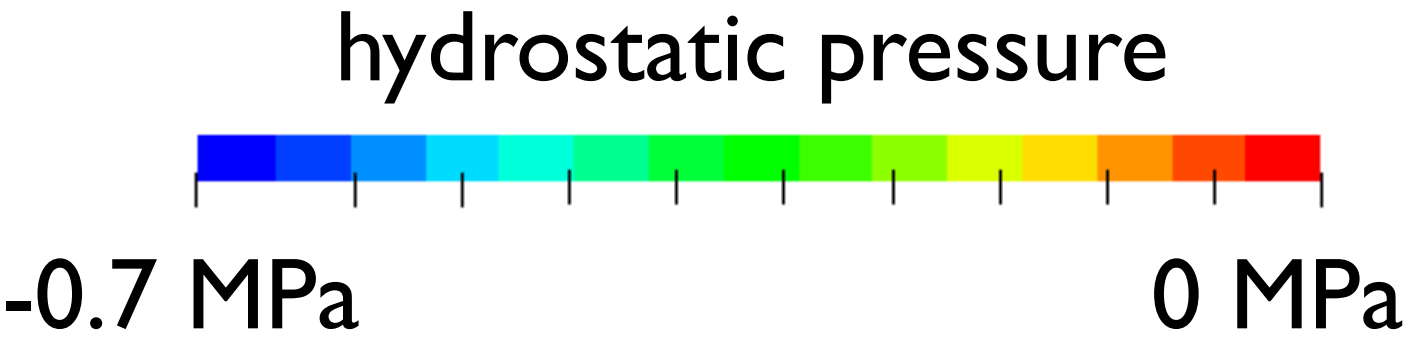


explicit

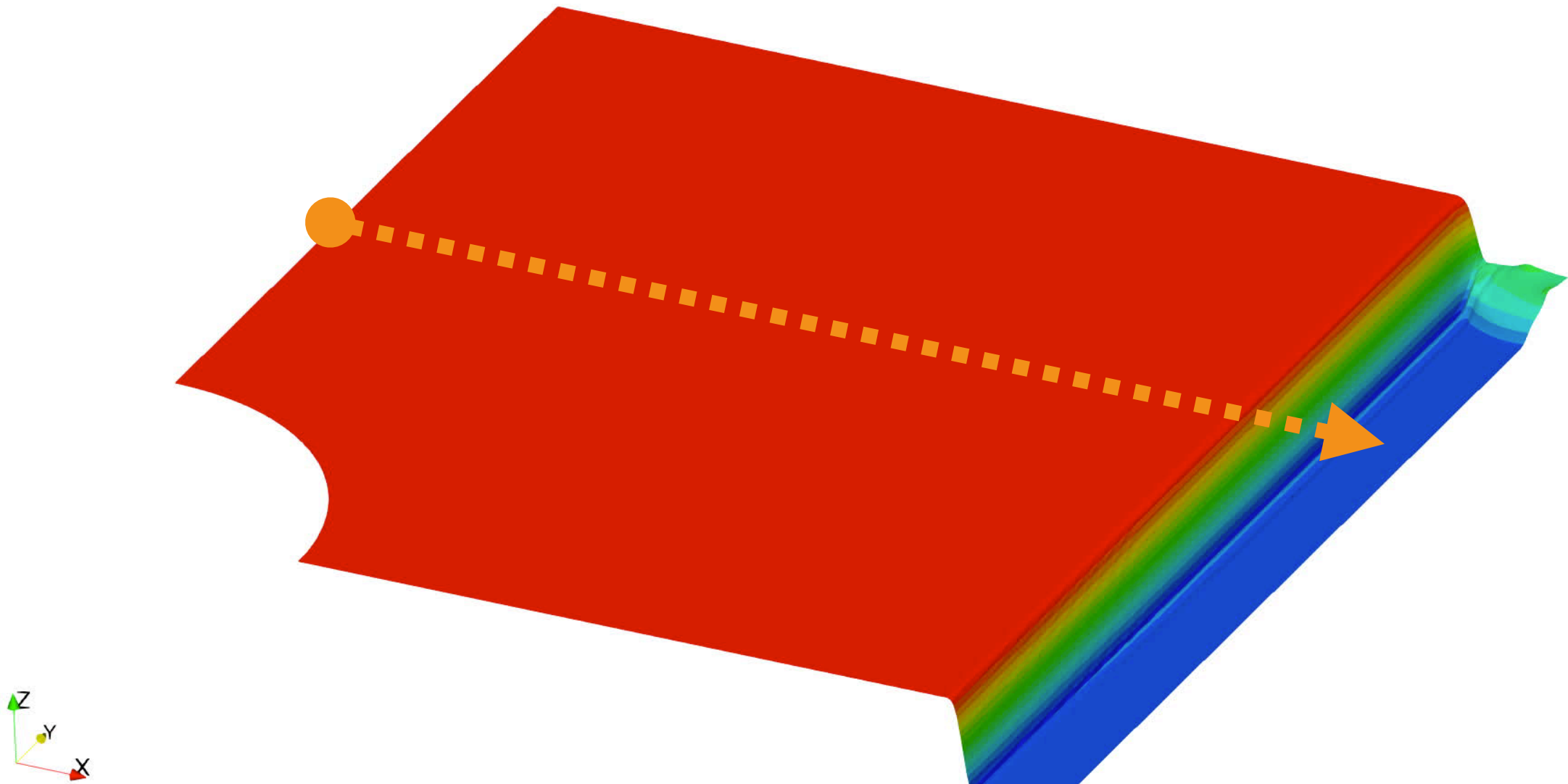
Time: 1.002 ms



implicit, segregated

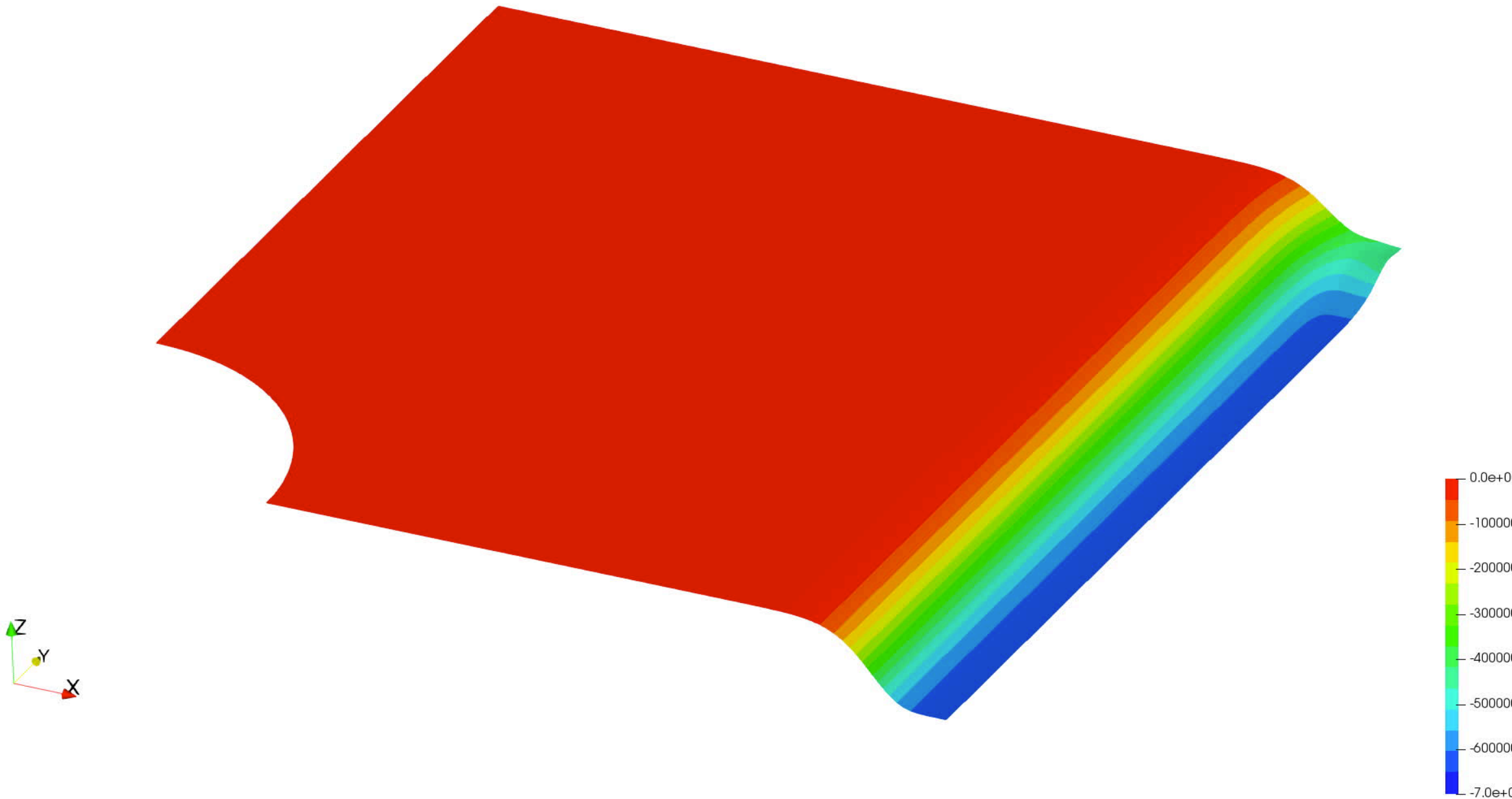


Time: 1.002 ms

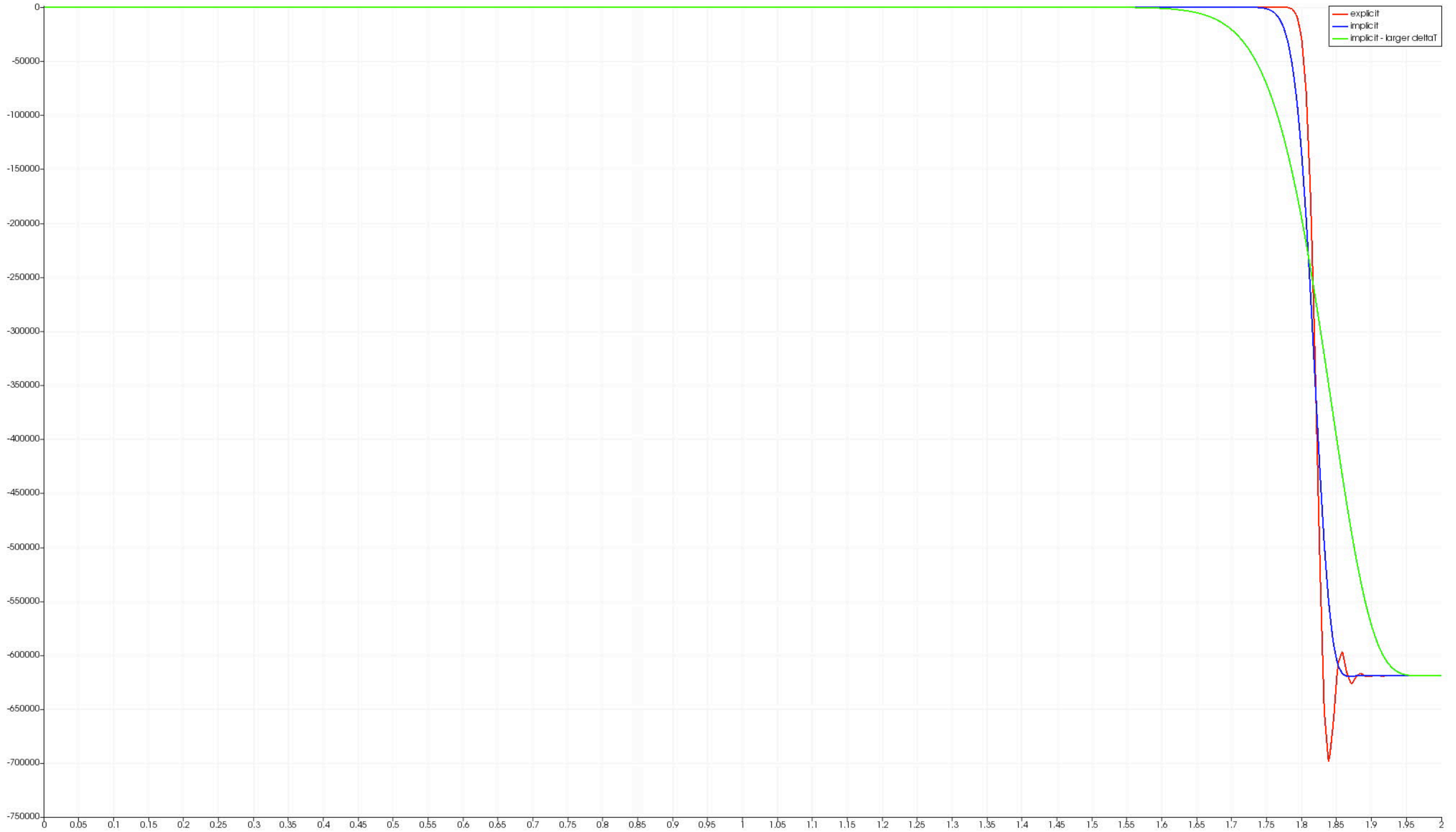


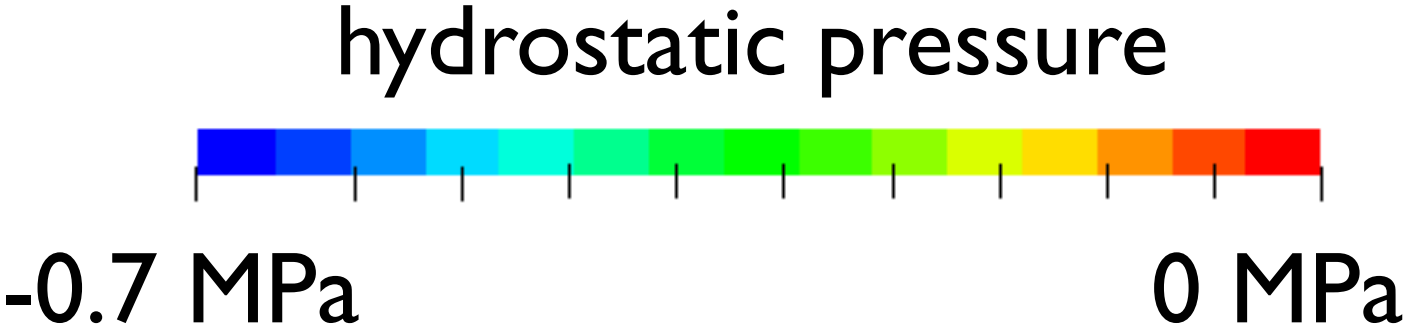
explicit

Time: 0.959 ms



implicit, segregated
 $\times 10 \text{ delta}T$

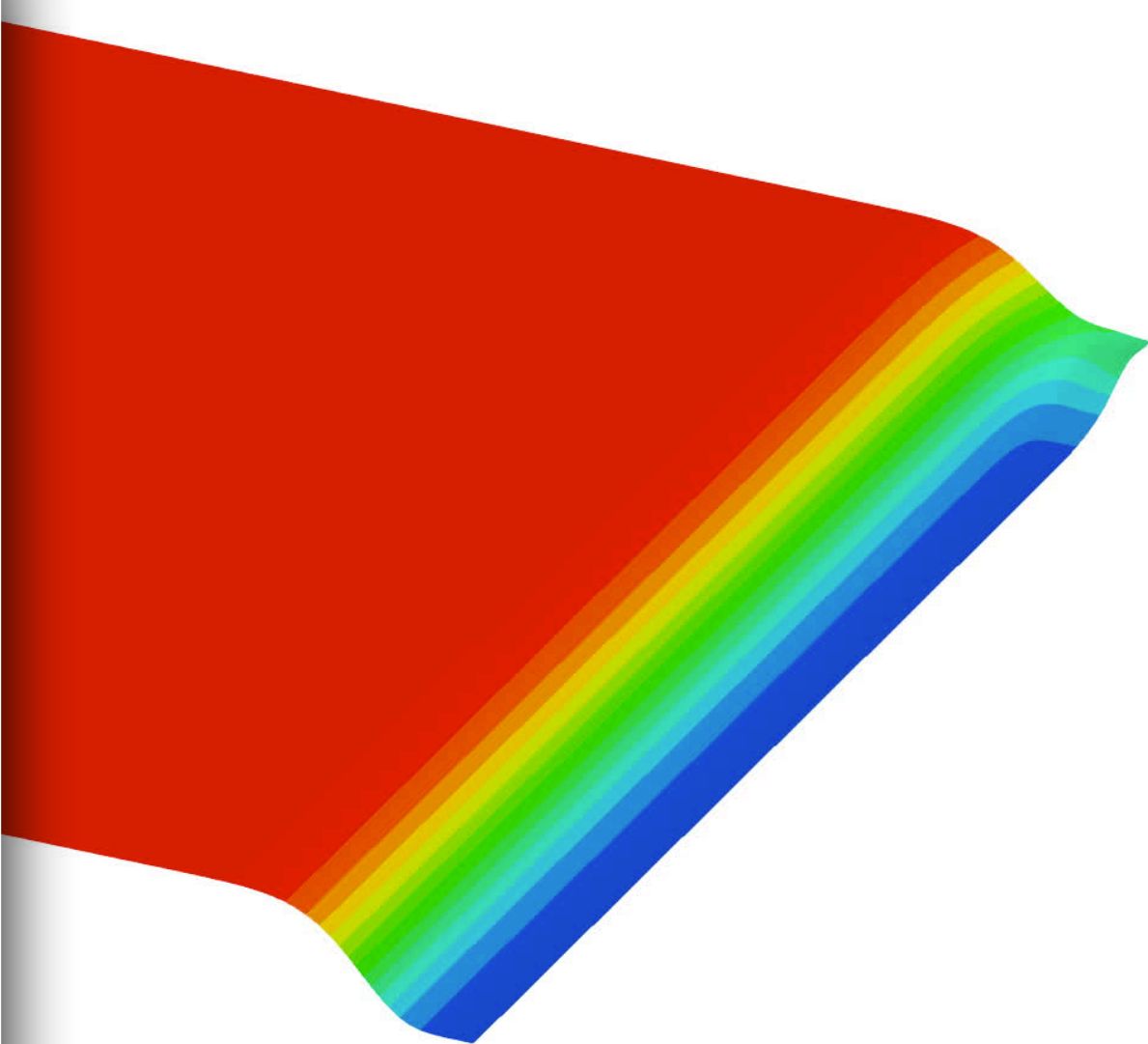
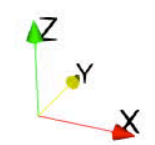




Time: 1.002 ms

Time: 0.959 ms

- **explicit**, interpolated gradients, JST stabilisation
 - time: 522 s
- **implicit, segregated**, interpolated gradients, RC stabilisation
 - time: 742 s
- **implicit, segregated**, larger deltaT
 - time: 354 s
- **implicit, coupled**, direct gradients, no stabilisation, iterative linear solver
 - time: 5,848 s



explicit

implicit, segregated
×10 deltaT

A large orange circle is positioned on the left side of the slide, partially cut off by the edge. The text 'Summary & Conclusions' is centered within this circle in white font.

Summary & Conclusions

Summary & Conclusions

- There have been significant developments in the finite volume methods for solid mechanics over the past 30 years
- Classification of the differing finite volume methodologies can be performed in a number of ways, including by: grid arrangement, implicit vs explicit, and by solution stabilisation
- A variety of finite volume solution methodologies for solid mechanics have been presented (but there are many more variants)
- In-depth comparison of the differing approaches is still required, in terms of accuracy, efficiency and robustness for a variety of cases.
- The finite element method is not the only approach suitable for solid mechanics

April 2019

Online International Meeting for Users of OpenFOAM
Published on YouTube

On the analysis of OpenFOAM finite volume solution methodologies for linear elasticity

Philip Cardiff



School of Mechanical and Materials Engineering
University College Dublin